

1. Introduction to Unions

What is a Union?

A **Union** is a user-defined data type in C similar to a structure.

The Core Difference: While structure members each get their own unique, isolated memory slots , all union members **share the exact same memory location**.

Consequently, a union variable can hold only **one member value at any given time**.

Defining a Union

To define a union, use the union keyword followed by a tag name.

```
union abc
{
    int a;
    char b;
}; -----> Semantically identical definition syntax to a struct
```

2. Memory Allocation: Structure vs. Union

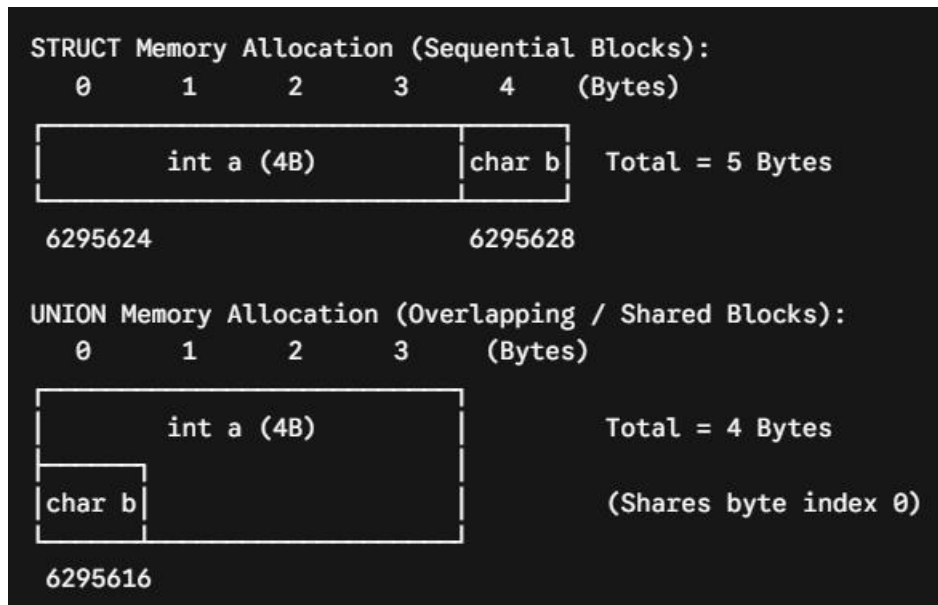
Because union members share memory , the compiler allocates a single block equal to the size of its **largest data member** , rather than summing up all the individual member sizes.

Comparative Memory Mapping

Assume a system where an int takes 4 bytes and a char takes 1 byte:

Data Type	Memory Strategy	Address Analysis Example	Total Size
struct abc	Each member receives a unique, sequential address.	a address: 6295624 b address: 6295628	5 Bytes (or more with padding)
union abc	All members point to the starting address block.	a address: 6295616	4 Bytes (Size of the largest member, int)

Data Type	Memory Strategy	Address Analysis Example	Total Size
		b address: 6295616	



3. Behavioral Mechanics and Member Corruption

Because a union uses a shared memory footprint, modifying one member **directly overwrites and corrupts** the values stored by other members. Only the most recently assigned variable will return accurate data.

Tracing Member Overwriting

Consider the following program snippet:

```
#include <stdio.h>
union abc
{
    int a;
    char b;
    float c;
}; -----> Largest member is float (4 bytes), so total size = 4 bytes
int main()
{
```

```

union abc u;

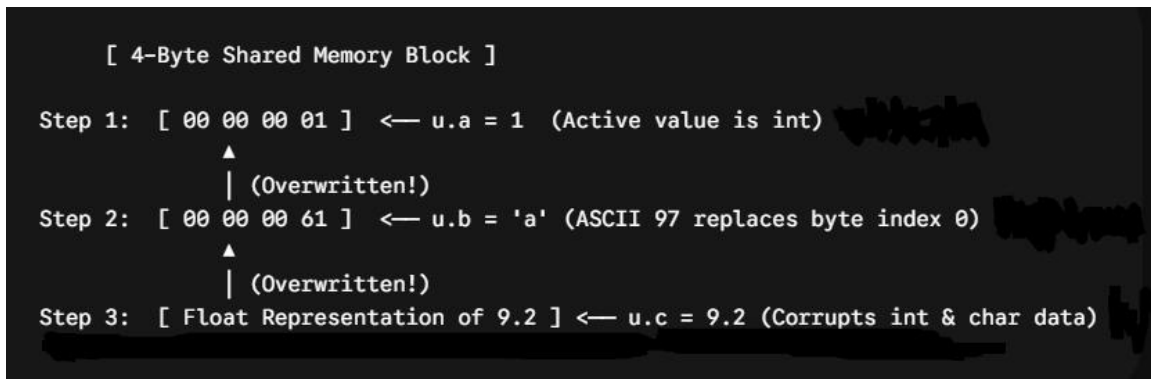
u.a = 1; -----> Step 1: Memory holds integer 1
u.b = 'a'; -----> Step 2: Memory is overwritten with character 'a' (ASCII 97)
u.c = 9.2; -----> Step 3: Memory is overwritten with float 9.2

printf("a = %d\n", u.a); -----> Outputs: Garbage value
printf("b = %c\n", u.b); -----> Outputs: Garbage value
printf("c = %f\n", u.c); -----> Outputs: 9.200000 (Valid!)

return 0;
}

```

Flow Diagram of the Overwrite Behavior



⚠ **Rule of Thumb:** To extract meaningful data from a union, you must access and read a member variable **immediately** after assigning it, before any other member allocations take place.

4. Summary: Structure vs. Union At-A-Glance

Feature	Structure (struct)	Union (union)
Keyword	struct	union
Memory Layout	Every member gets an independent address.	All members share the same starting address.
Total Size	Sum of sizes of all members	Size of the single largest

Feature	Structure (struct)	Union (union)
	(+ padding bytes).	member.
Member Alteration	Changing one member does not affect others.	Changing one member alters all other member profiles.
Simultaneous Use	All members can be accessed concurrently.	Only one member can be accessed safely at a time.